

LAXMI CHARITABLE TRUST'S
SHETH L.U.J COLLEGE OF ARTS & SIR M.V. COLLEGE OF SCIENCE AND COMMERCE
DEPARTMENT OF INFORMATION TECHNOLOGY

BUSINESS ANALYTICS PROJECT

WALMART BUSINESS ANALYTICS
Sales, Profitability, Customer Behaviour and Decision Analysis

Submitted By:

Name: Soham Sachin Acharekar

Roll No.: T001

Class: TY B.Sc. IT

Division / Batch: 01

Subject: Business Analytics

Academic Year: 2026–2027

Table of Contents

1. Introduction
2. Problem Statement and Objectives
3. Dataset and Tools
4. Setup, Data Loading and Preprocessing
5. Descriptive Statistics
6. Exploratory Data Analysis
7. Correlation Analysis
8. Regression Analysis
9. Probability Analysis
10. Sampling and Central Limit Theorem
11. Confidence Interval
12. Bootstrapping
13. Hypothesis Testing
14. A/B Testing
15. Time-Series Analysis
16. Simulation
17. ANOVA
18. Factorial Experiment
19. Decision Making / Expected Value
20. Optimization
21. Final Business Summary
22. Business Recommendations
23. Conclusion
24. References

1. Introduction

Business Analytics is the use of data, statistical methods and visualization to understand business performance and support better decisions. This project analyzes a Walmart retail business dataset using Python and applies exploratory data analysis, descriptive statistics, correlation, regression, probability, sampling, confidence intervals, bootstrapping, hypothesis testing, A/B testing, time-series analysis, simulation, ANOVA, factorial experiments, decision analysis and optimization.

The project converts transactional business data into useful insights related to sales, profit, discounts, product categories, regions, sales channels, customer ratings and resource allocation.

2. Problem Statement and Objectives

Retail businesses generate large amounts of data across products, customers, regions, sales channels and dates. Analysing this information manually can make it difficult to identify important trends and make data-driven decisions.

- Understand sales and profit performance across regions and product categories.
- Study relationships among sales, quantity, discount, profit, shipping cost and customer rating.
- Build regression models to understand sales relationships.
- Calculate business probabilities and sampling distributions.
- Estimate confidence intervals and use bootstrapping.

- Test business assumptions using hypothesis testing.
- Compare promotional alternatives using A/B testing.
- Study monthly sales trends using time-series analysis.
- Evaluate uncertain demand scenarios using simulation.
- Compare groups using ANOVA.
- Study the interaction of sales method and sales volume using a factorial experiment.
- Select business alternatives using expected value and optimize limited resources.

3. Dataset and Tools

The main dataset is Walmart_Business_Analytics_Master_Dataset.csv. It contains 10,000 business transaction records and 17 columns. Supporting academic datasets are provided for A/B testing, simulation, factorial experiment, decision making and optimization.

Item	Details	Purpose
Language	Python	Business analytics and statistics
Environment	Jupyter Notebook / VS Code	Execution
Libraries	Pandas, NumPy, Matplotlib, Seaborn	Data processing and visualization
Libraries	SciPy, Scikit-learn, Statsmodels	Statistics and machine learning
Library	PuLP	Optimization
Data format	CSV	Dataset storage

Main Dataset Columns:

Order_ID, Order_Date, Customer_ID, Customer_Segment, Product_Category, Product, Region, State, Sales, Quantity, Discount, Profit, Shipping_Cost, Ship_Mode, Payment_Mode, Sales_Channel, Customer_Rating.

4. Setup, Data Loading and Preprocessing

Objective: Import libraries and load the Walmart master dataset.

The dataset is loaded from the Dataset folder. Basic checks are performed for shape, column names, missing values, duplicate records and data types. Order_Date is converted to datetime before time-series analysis.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

print("Libraries imported successfully!")
```

✓ 2.3s

Libraries imported successfully!

```
df = pd.read_csv("Walmart_Business_Analytics_Master_Dataset.csv")
```

```
print("Dataset loaded successfully!")
print("Rows:", df.shape[0])
print("Columns:", df.shape[1])
display(df.head())
```

✓ 0.1s

Dataset loaded successfully!

Rows: 10000

Columns: 17

	Order_ID	Order_Date	Customer_ID	Customer_Segment	Product_Category	Product	Region	State	Sales	Quantity	Discount	Profit	Shipping_Cost	Ship_Mode	Payment_Mode	Sales_Channel	Customer_Rating
0	ORD100000	2024-11-07	CUST01899	Corporate	Furniture	Desk	East	New York	1058.65	5	0.1530	243.95	14.58	Second Class	Debit Card	Online	4.28
1	ORD100001	2023-07-23	CUST02199	Corporate	Furniture	Table	South	Florida	1482.13	4	0.0561	304.82	13.69	Standard	Credit Card	Store	3.53
2	ORD100002	2023-09-08	CUST00411	Consumer	Technology	Monitor	Central	Ohio	2323.86	8	0.1229	256.61	41.00	Standard	Credit Card	Online	3.09
3	ORD100003	2025-12-30	CUST02202	Corporate	Office Supplies	File Folder	Central	Illinois	773.85	8	0.1587	35.82	37.41	First Class	Debit Card	Online	4.09
4	ORD100004	2023-09-12	CUST00754	Consumer	Office Supplies	Notebook	West	California	666.59	10	0.0994	84.27	36.39	Standard	Credit Card	Online	4.18

```
df.info()
```

✓ 0.1s

```
<class 'pandas.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 17 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Order_ID              10000 non-null  str
1   Order_Date            10000 non-null  str
2   Customer_ID           10000 non-null  str
3   Customer_Segment      10000 non-null  str
4   Product_Category      10000 non-null  str
5   Product               10000 non-null  str
6   Region                10000 non-null  str
7   State                 10000 non-null  str
8   Sales                 10000 non-null  float64
9   Quantity              10000 non-null  int64
10  Discount              10000 non-null  float64
11  Profit                10000 non-null  float64
12  Shipping_Cost         10000 non-null  float64
13  Ship_Mode             10000 non-null  str
14  Payment_Mode          10000 non-null  str
15  Sales_Channel         10000 non-null  str
16  Customer_Rating       10000 non-null  float64
dtypes: float64(5), int64(1), str(11)
memory usage: 2.2 MB
```

```
print("Missing values:")
display(df.isnull().sum())
```

✓ 0.0s

Missing values:

```
Order_ID      0
Order_Date    0
Customer_ID   0
Customer_Segment  0
Product_Category  0
Product       0
Region        0
State         0
Sales         0
Quantity      0
Discount      0
Profit        0
Shipping_Cost 0
Ship_Mode     0
Payment_Mode  0
Sales_Channel 0
Customer_Rating 0
dtype: int64
```

```
print("Duplicate records:", df.duplicated().sum())
```

✓ 0.0s

Duplicate records: 0

```
display(df.dtypes)
```

✓ 0.0s

```
Order_ID      str
Order_Date    str
Customer_ID   str
Customer_Segment str
Product_Category str
Product       str
Region        str
State         str
Sales         float64
Quantity      int64
Discount      float64
Profit        float64
Shipping_Cost float64
Ship_Mode     str
Payment_Mode  str
Sales_Channel str
Customer_Rating float64
dtype: object
```

5. Descriptive Statistics

Objective: Calculate mean, median, minimum, maximum and standard deviation for important numeric variables.

Descriptive statistics summarize Sales, Quantity, Discount, Profit, Shipping Cost and Customer Rating.

```
columns = [  
    "Sales",  
    "Quantity",  
    "Discount",  
    "Profit",  
    "Shipping_Cost",  
    "Customer_Rating"  
]
```

```
display(df[columns].describe())
```

✓ 0.0s

	Sales	Quantity	Discount	Profit	Shipping_Cost	Customer_Rating
count	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000
mean	1072.049419	5.477300	0.106038	176.531654	24.504236	3.952813
std	1328.016091	2.876024	0.055677	262.310566	10.609915	0.461951
min	18.180000	1.000000	0.000800	-26.950000	9.200000	1.660000
25%	231.405000	3.000000	0.063500	17.347500	16.020000	3.640000
50%	474.600000	5.000000	0.097500	62.850000	22.705000	3.960000
75%	1456.180000	8.000000	0.139400	225.550000	31.172500	4.270000
max	7761.440000	10.000000	0.357200	1859.870000	64.860000	5.000000

6. Exploratory Data Analysis

EDA uses tables and visualizations to discover patterns, trends, differences and possible relationships in the business data.

6.1 Total Sales by Region

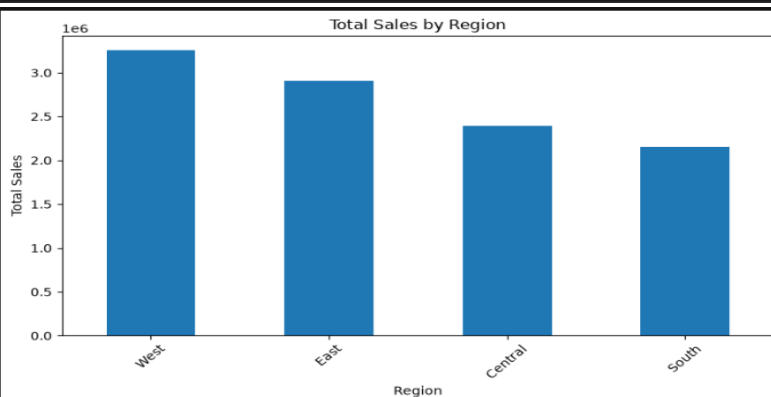
Compare total sales across the four business regions.

```
# 6.1 Total Sales by Region
```

```
sales_region = (  
    df.groupby("Region")["Sales"]  
    .sum()  
    .sort_values(ascending=False)  
)
```

```
sales_region.plot(kind="bar", figsize=(8, 5))  
plt.title("Total Sales by Region")  
plt.xlabel("Region")  
plt.ylabel("Total Sales")  
plt.xticks(rotation=45)  
plt.tight_layout()  
plt.show()
```

✓ 1.3s



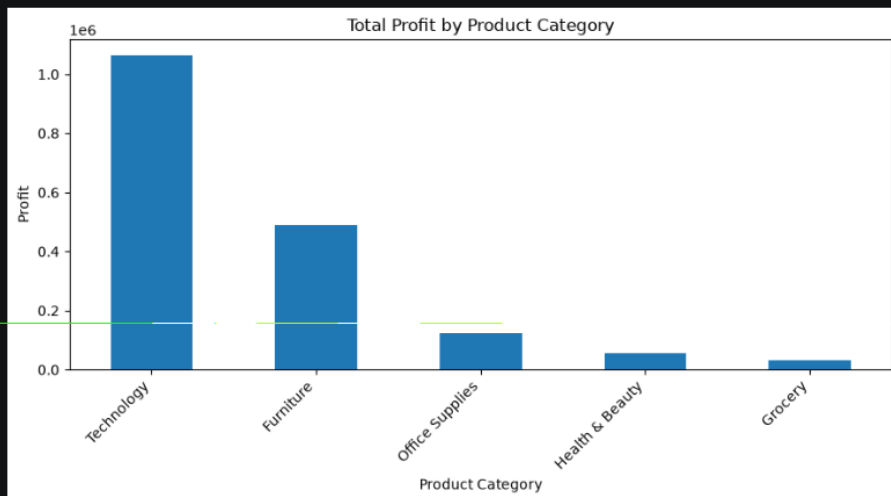
6.2 Profit by Product Category

Identify product categories that generate the highest and lowest total profit.

```
profit_category = (  
    df.groupby("Product_Category")["Profit"]  
    .sum()  
    .sort_values(ascending=False)  
)
```

```
profit_category.plot(kind="bar", figsize=(9, 5))  
plt.title("Total Profit by Product Category")  
plt.xlabel("Product Category")  
plt.ylabel("Profit")  
plt.xticks(rotation=45, ha="right")  
plt.tight_layout()  
plt.show()
```

✓ 0.2s

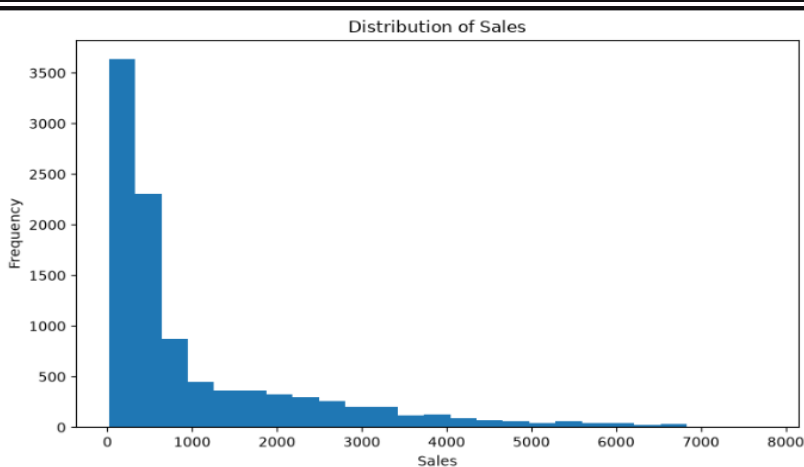


6.3 Sales Histogram

Study the distribution and frequency of transaction sales.

```
plt.figure(figsize=(8, 5))  
plt.hist(df["Sales"].dropna(), bins=25)  
plt.title("Distribution of Sales")  
plt.xlabel("Sales")  
plt.ylabel("Frequency")  
plt.tight_layout()  
plt.show()
```

✓ 0.2s

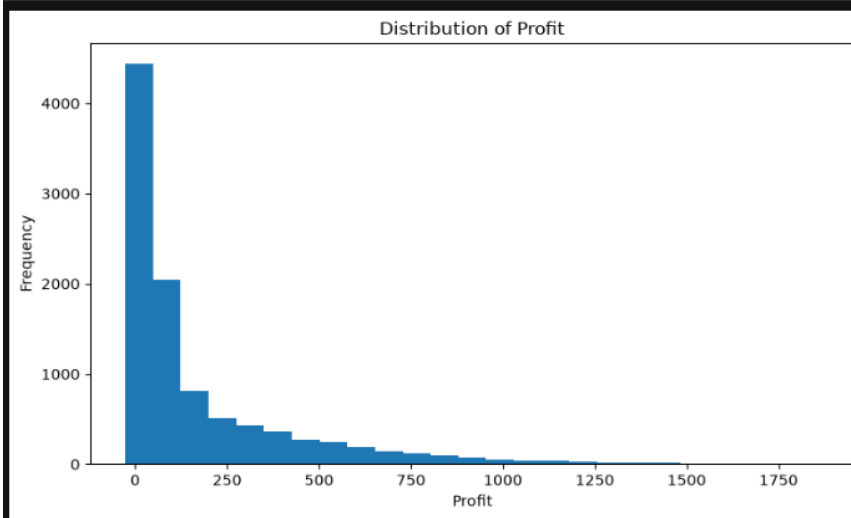


6.4 Profit Histogram

Study the distribution and spread of profit.

```
plt.figure(figsize=(8, 5))
plt.hist(df["Profit"].dropna(), bins=25)
plt.title("Distribution of Profit")
plt.xlabel("Profit")
plt.ylabel("Frequency")
plt.tight_layout()
plt.show()
```

✓ 0.2s

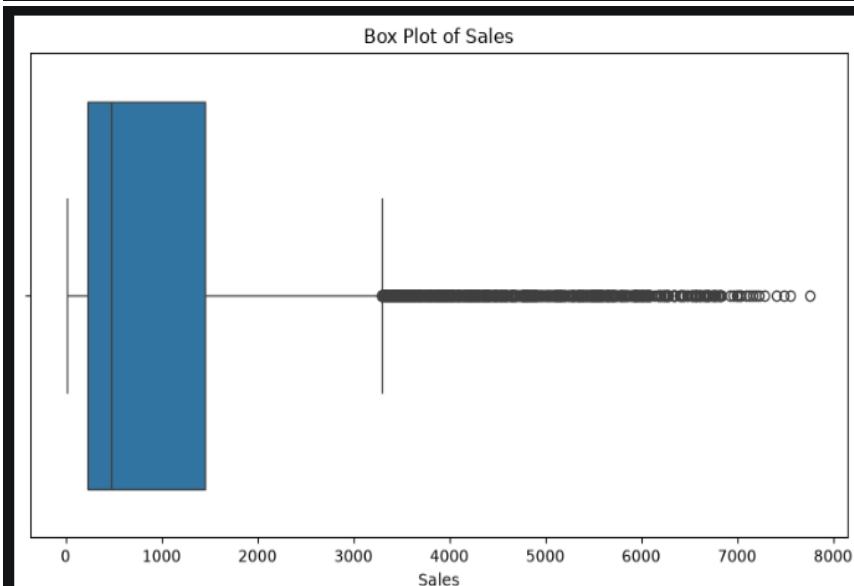


6.5 Sales Box Plot

Identify the median, spread and possible outliers in sales.

```
plt.figure(figsize=(8, 5))
sns.boxplot(x=df["Sales"])
plt.title("Box Plot of Sales")
plt.tight_layout()
plt.show()
```

✓ 0.3s



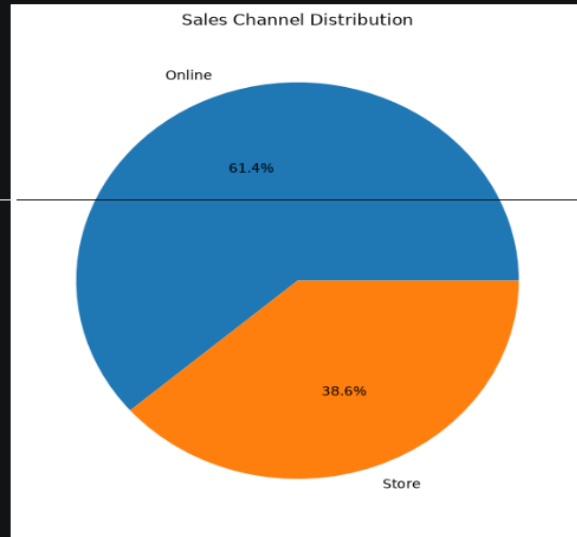
6.6 Sales Channel Distribution

Compare the proportion of Online and Store transactions.

```
sales_channel = df["Sales_Channel"].value_counts()

plt.figure(figsize=(7, 7))
plt.pie(
    sales_channel,
    labels=sales_channel.index,
    autopct="%1.1f%%"
)
plt.title("Sales Channel Distribution")
plt.show()
```

✓ 0.1s



6.7 Sales vs Profit

Use a scatter plot to study whether higher sales are associated with higher profit.

```
plt.figure(figsize=(8, 5))
plt.scatter(
    df["Sales"],
    df["Profit"],
    alpha=0.5
)
plt.title("Sales vs Profit")
plt.xlabel("Sales")
plt.ylabel("Profit")
plt.tight_layout()
plt.show()
```

✓ 0.3s



7. Correlation Analysis

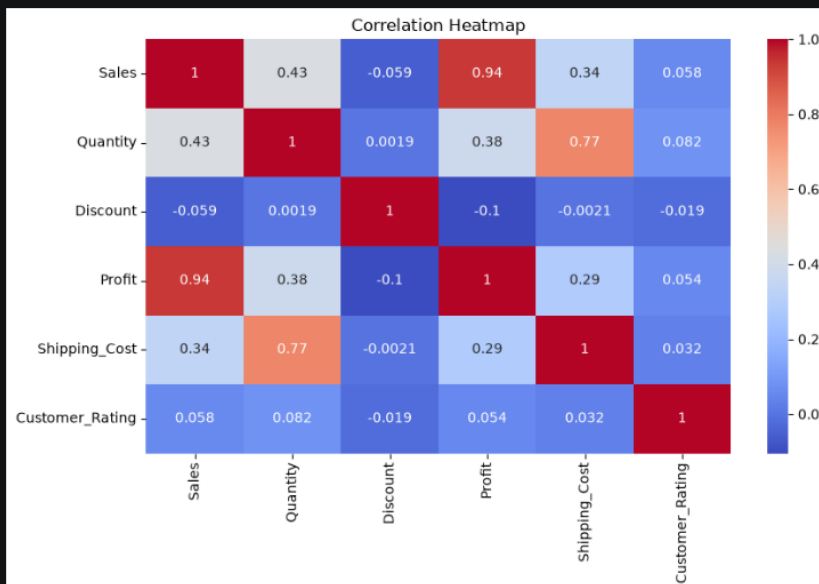
Objective: Measure relationships between Sales, Quantity, Discount, Profit, Shipping Cost and Customer Rating.

The correlation matrix and heatmap show the strength and direction of relationships among numeric business variables.

```
numeric_columns = [  
    "Sales",  
    "Quantity",  
    "Discount",  
    "Profit",  
    "Shipping_Cost",  
    "Customer_Rating"  
]  
  
correlation = df[numeric_columns].corr()  
  
display(correlation)  
  
plt.figure(figsize=(9, 6))  
sns.heatmap(  
    correlation,  
    annot=True,  
    cmap="coolwarm"  
)  
plt.title("Correlation Heatmap")  
plt.tight_layout()  
plt.show()
```

✓ 0.3s

	Sales	Quantity	Discount	Profit	Shipping_Cost	Customer_Rating
Sales	1.000000	0.433040	-0.059262	0.936876	0.339779	0.058475
Quantity	0.433040	1.000000	0.001915	0.381222	0.771686	0.081938
Discount	-0.059262	0.001915	1.000000	-0.104639	-0.002111	-0.018654
Profit	0.936876	0.381222	-0.104639	1.000000	0.285491	0.054209
Shipping_Cost	0.339779	0.771686	-0.002111	0.285491	1.000000	0.031794
Customer_Rating	0.058475	0.081938	-0.018654	0.054209	0.031794	1.000000



8. Regression Analysis

Objective: Build a simple linear regression model using Quantity as the independent variable and Sales as the dependent variable.

The model estimates Sales from Quantity. Coefficient, intercept, R-squared, MSE and RMSE are reported, followed by a regression line.

```

from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

regression_df = df[["Quantity", "Sales"]].dropna()

X = regression_df[["Quantity"]]
y = regression_df["Sales"]

model = LinearRegression()
model.fit(X, y)

prediction = model.predict(X)

print("Coefficient:", model.coef_[0])
print("Intercept:", model.intercept_)
print("R-squared:", r2_score(y, prediction))
print("MSE:", mean_squared_error(y, prediction))
print("RMSE:", np.sqrt(mean_squared_error(y, prediction)))

```

✓ 1.2s

```

plot_df = regression_df.sort_values("Quantity")

plt.figure(figsize=(8, 5))
plt.scatter(
    regression_df["Quantity"],
    regression_df["Sales"],
    alpha=0.5,
    label="Actual"
)

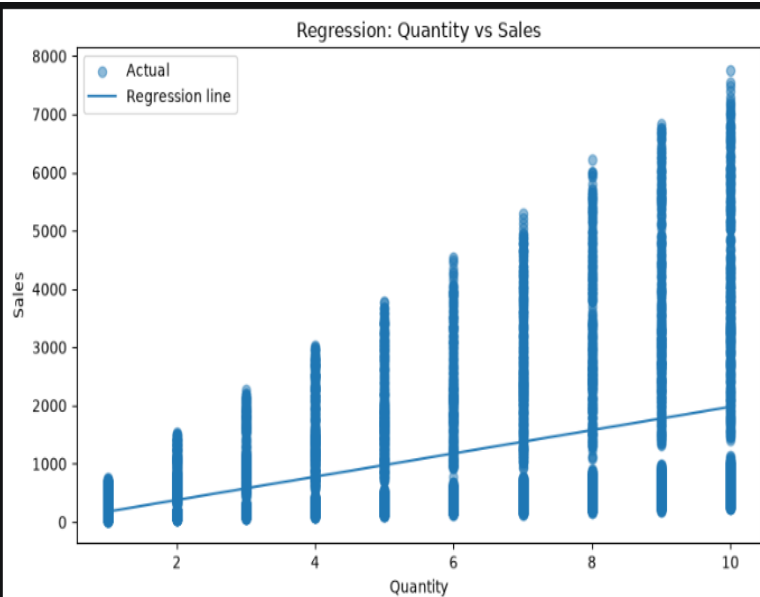
plt.plot(
    plot_df["Quantity"],
    model.predict(plot_df[["Quantity"]]),
    label="Regression line"
)

plt.title("Regression: Quantity vs Sales")
plt.xlabel("Quantity")
plt.ylabel("Sales")
plt.legend()
plt.tight_layout()
plt.show()

```

✓ 0.4s

Coefficient: 199.95811293733595
 Intercept: -23.181152991669933
 R-squared: 0.18752375107950392
 MSE: 1432761.545287601
 RMSE: 1196.9801774831533



9. Probability Analysis

Objective: Calculate the probability of Online sales and Technology-category transactions.

Probability is calculated as favourable records divided by total records.

```
total_records = len(df)

online_records = (
    df["Sales_Channel"]
    .astype(str)
    .str.lower()
    .eq("online")
).sum()

print(
    "Probability of Online sales:",
    online_records / total_records
    if total_records else 0
)

technology_records = (
    df["Product_Category"] == "Technology"
).sum()

print(
    "Probability of Technology category:",
    technology_records / total_records
    if total_records else 0
)
```

✓ 0.0s

Probability of Online sales: 0.6135
Probability of Technology category: 0.2171

10. Sampling and Central Limit Theorem

Objective: Draw repeated samples of sizes 30, 50 and 100 and compare their sample means and standard errors.

Repeated sampling demonstrates how sample means behave as sample size increases. The sampling distribution for n=100 is plotted.

```
sample_sizes = [30, 50, 100]

sales = df["Sales"].dropna()

for size in sample_sizes:

    sample_means = []

    for _ in range(100):
        sample_means.append(
            sales.sample(size, replace=True).mean()
        )

    print("Sample size:", size)
    print(
        "Mean of sample means:",
        round(np.mean(sample_means), 2)
    )

    print(
        "Standard error:",
        round(np.std(sample_means, ddof=1), 2)
    )
```

✓ 0.0s

```

sales = df["Sales"].dropna()

sample_means = [
    sales.sample(100, replace=True).mean()
    for _ in range(100)
]

plt.figure(figsize=(8, 5))
plt.hist(sample_means, bins=20)
plt.title("Sampling Distribution of Mean Sales (n=100)")
plt.xlabel("Sample Mean")
plt.ylabel("Frequency")
plt.tight_layout()
plt.show()

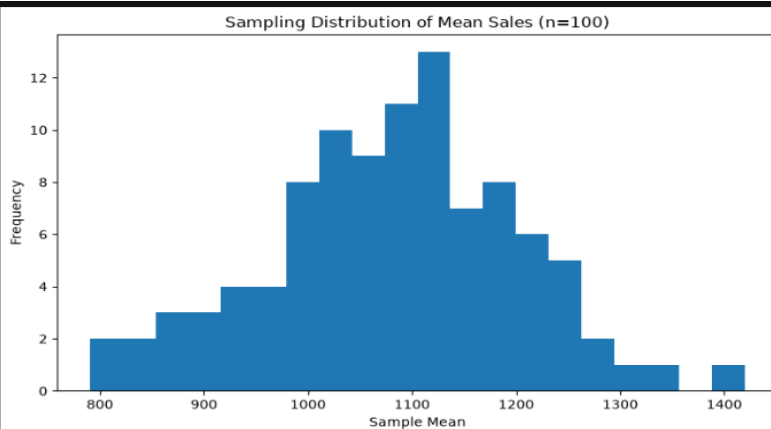
```

✓ 0.2s

```

Sample size: 30
Mean of sample means: 1072.49
Standard error: 223.57
Sample size: 50
Mean of sample means: 1055.49
Standard error: 170.51
Sample size: 100
Mean of sample means: 1073.29
Standard error: 129.92

```



11. Confidence Interval

Objective: Calculate a 95% confidence interval for mean Sales.

The t-based interval provides a range of plausible values for the population mean Sales.

```

from scipy import stats

sales = df["Sales"].dropna()

mean = sales.mean()
n = len(sales)

standard_error = stats.sem(sales)

lower, upper = stats.t.interval(
    0.95,
    df=n-1,
    loc=mean,
    scale=standard_error
)

print("Mean Sales:", round(mean, 2))

print(
    "95% Confidence Interval:",
    (round(lower, 2), round(upper, 2))
)

```

✓ 0.0s

```

Mean Sales: 1072.05
95% Confidence Interval: (np.float64(1046.02), np.float64(1098.08))

```

12. Bootstrapping

Objective: Use 1,000 bootstrap resamples to estimate uncertainty of mean Sales.

The bootstrap distribution is used to calculate a 95% percentile confidence interval.

```
sales = df["Sales"].dropna().to_numpy()

bootstrap_means = []

for _ in range(1000):
    bootstrap_sample = np.random.choice(
        sales,
        size=len(sales),
        replace=True
    )

    bootstrap_means.append(
        bootstrap_sample.mean()
    )

lower, upper = np.percentile(
    bootstrap_means,
    [2.5, 97.5]
)

print(
    "Bootstrap mean:",
    round(np.mean(bootstrap_means), 2)
)

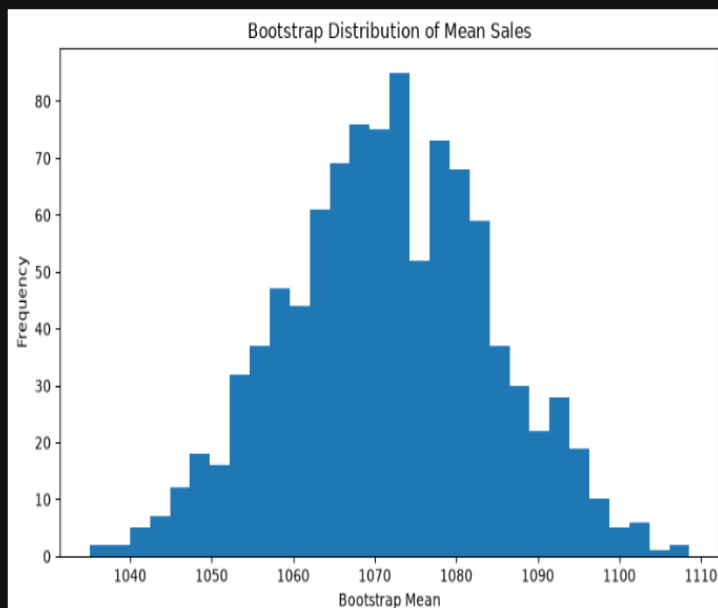
print(
    "Bootstrap 95% confidence interval:",
    (round(lower, 2), round(upper, 2))
)

plt.figure(figsize=(8, 5))
plt.hist(bootstrap_means, bins=30)
plt.title("Bootstrap Distribution of Mean Sales")
plt.xlabel("Bootstrap Mean")
plt.ylabel("Frequency")
plt.tight_layout()
plt.show()
```

✓ 0.6s

Bootstrap mean: 1071.74

Bootstrap 95% confidence interval: (np.float64(1046.94), np.float64(1096.19))



```
data = df["Sales"]
bootstrap_means = []
for _ in range(1000):
    bootstrap_sample = np.random.choice(
        data,
        size=len(data),
        replace=True
    )
    bootstrap_means.append(
        bootstrap_sample.mean()
    )
lower, upper = np.percentile(
    bootstrap_means,
    [2.5, 97.5]
)
print(
    "Bootstrap mean:",
    round(np.mean(bootstrap_means), 2)
)
print(
    "Bootstrap 95% confidence interval:",
    (round(lower, 2), round(upper, 2))
)
plt.figure(figsize=(8, 5))
plt.hist(bootstrap_means, bins=30)
plt.title("Bootstrap Distribution of Mean Sales")
plt.xlabel("Bootstrap Mean")
plt.ylabel("Frequency")
plt.tight_layout()
plt.show()
```

13. Hypothesis Testing

Objective: Test whether average Sales is significantly different from 500.

H_0 : Mean Sales = 500. H_1 : Mean Sales is different from 500. $\alpha=0.05$. If $p<0.05$, reject H_0 .

```
from scipy.stats import ttest_1samp

sales = df["Sales"].dropna()

benchmark = 500

result = ttest_1samp(
    sales,
    popmean=benchmark
)

print("H0: Mean Sales = 500")
print("H1: Mean Sales is different from 500")
print("t-statistic:", result.statistic)
print("p-value:", result.pvalue)

print(
    "Decision:",
    "Reject H0"
    if result.pvalue < 0.05
    else "Fail to reject H0"
)
```

✓ 0.0s

```
H0: Mean Sales = 500
H1: Mean Sales is different from 500
t-statistic: 43.075488545067884
p-value: 0.0
Decision: Reject H0
```

14. A/B Testing

Objective: Compare conversion rates between Variant A and Variant B using a two-proportion z-test.

The academic A/B dataset compares conversions and determines whether the difference in conversion rates is statistically significant.

```
ab = pd.read_csv(
    "Walmart_AB_Testing_Dataset.csv"
)

display(ab.head())

display(
    ab.groupby("Variant")["Converted"]
    .agg(["sum", "count", "mean"])
)
```

✓ 0.0s

```
from statsmodels.stats.proportion import proportions_ztest

summary = (
    ab.groupby("Variant")["Converted"]
    .agg(["sum", "count"])
)

z_stat, p_value = proportions_ztest(
    summary["sum"].to_numpy(),
    summary["count"].to_numpy()
)

print("Z-statistic:", z_stat)
print("P-value:", p_value)

print(
    "Decision:",
    "Significant difference"
    if p_value < 0.05
    else "No significant difference detected"
)
```

✓ 0.0s

	Customer_ID	Variant	Converted
0	AB10000	B	0
1	AB10001	B	0
2	AB10002	B	1
3	AB10003	A	0
4	AB10004	B	0

	sum	count	mean
Variant			
A	108	1003	0.107677
B	140	997	0.140421

Z-statistic: -2.221548466454159
P-value: 0.02631383584986329
Decision: Significant difference

15. Time-Series Analysis

Time-series analysis examines how Walmart sales change over time.

15.1 Monthly Sales

Convert Order_Date to datetime and calculate total Sales for each month.

```
df["Order_Date"] = pd.to_datetime(
    df["Order_Date"],
    errors="coerce"
)

dated_df = df.dropna(
    subset=["Order_Date"]
).copy()

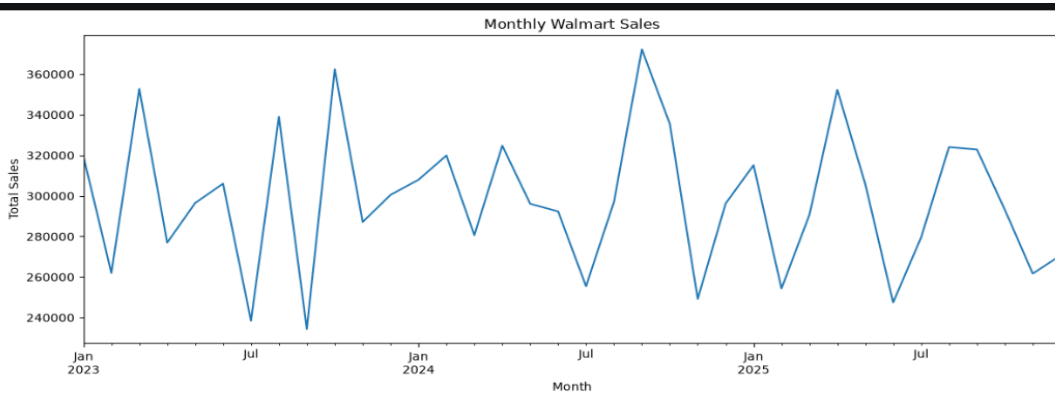
monthly_sales = (
    dated_df
    .groupby(
        dated_df["Order_Date"].dt.to_period("M")
    )["Sales"]
    .sum()
)

monthly_sales.index = (
    monthly_sales.index.to_timestamp()
)

monthly_sales.plot(figsize=(12, 5))

plt.title("Monthly Walmart Sales")
plt.xlabel("Month")
plt.ylabel("Total Sales")
plt.tight_layout()
plt.show()
```

✓ 0.3s



```
df["Order_Date"] = pd.to_datetime(
    df["Order_Date"],
    errors="coerce"
)

dated_df = df.dropna(
    subset=["Order_Date"]
).copy()

monthly_sales = (
    dated_df
    .groupby(
        dated_df["Order_Date"].dt.to_period("M")
    )["Sales"]
    .sum()
)

monthly_sales.index = (
    monthly_sales.index.to_timestamp()
)
```

15.2 Three-Month Moving Average

Smooth short-term fluctuations using a three-month moving average to identify the general sales trend.



16. Simulation

Objective: Compare Low, Normal and High demand scenarios using simulated demand, sales and profit.

Average demand, sales and profit are compared across scenarios to support planning under uncertainty.



Available columns: ['Scenario', 'Demand_Units', 'Average_Price', 'Sales', 'Profit']

	Scenario	Demand_Units	Average_Price	Sales	Profit
0	Normal	105	92.03	9644.47	1831.78
1	Normal	117	104.90	12264.04	1591.01
2	High	140	86.39	12087.99	2988.87
3	Normal	108	102.86	11126.15	1583.21
4	Normal	79	50.49	4001.63	863.20

	Demand_Units	Average_Price	Sales	Profit
Scenario				
High	149.785915	78.024507	11712.958451	2352.071775
Low	70.356955	77.629528	5471.856693	1090.192782
Normal	109.073298	77.118887	8406.137291	1663.107762

17. ANOVA

Objective: Test whether average Profit differs significantly between Online and Store sales channels.

One-way ANOVA compares the mean Profit of the two sales-channel groups. H_0 states equal means; H_1 states at least one mean differs. $\alpha=0.05$.

```
from scipy.stats import f_oneway

groups = []
group_names = []

for name, group in df.groupby("Sales_Channel"):
    values = group["Profit"].dropna()

    if len(values) > 1:
        groups.append(values)
        group_names.append(name)

print("Groups tested:", group_names)

if len(groups) >= 2:
    f_stat, p_value = f_oneway(*groups)

    print("F-statistic:", f_stat)
    print("P-value:", p_value)

    print(
        "Decision:",
        "Significant difference"
        if p_value < 0.05
        else "No significant difference detected"
    )
else:
    print("At least two groups are required.")
```

✓ 0.0s

```
Groups tested: ['Online', 'Store']
F-statistic: 1.6051487786677547
P-value: 0.20520458880856676
Decision: No significant difference detected
```

18. Factorial Experiment

Objective: Study the effect of Sales Method and Sales Volume Group on Operating Profit.

The 2x3 experiment contains Online/Store and Low/Medium/High volume groups. A mean table and interaction plot evaluate the factors.

```

factorial = pd.read_csv(
    "Walmart_Factorial_Experiment_Dataset.csv"
)

print(
    "Available columns:",
    factorial.columns.tolist()
)

display(factorial.head())

if {
    "Sales_Method",
    "Sales_Volume_Group",
    "Operating_Profit"
}.issubset(factorial.columns):

    display(
        factorial
        .groupby(
            ["Sales_Method", "Sales_Volume_Group"]
        )["Operating_Profit"]
        .mean()
        .unstack()
    )

    sns.pointplot(
        data=factorial,
        x="Sales_Volume_Group",
        y="Operating_Profit",
        hue="Sales_Method"
    )

    plt.title("Factorial Experiment")
    plt.tight_layout()
    plt.show()

else:
    print(
        "Expected columns not found; check the column list above."
    )

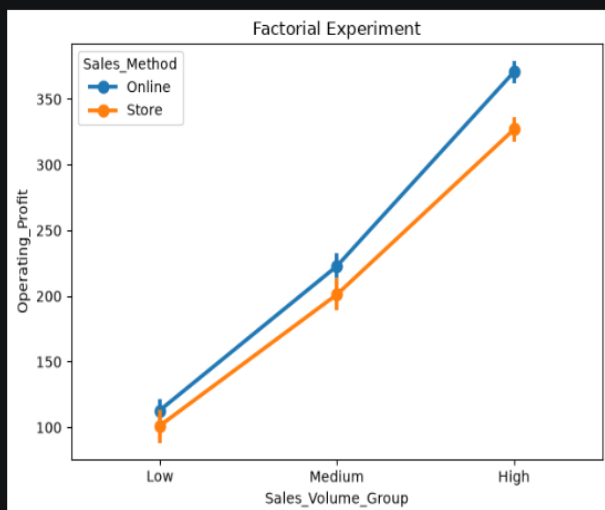
```

✓ 0.4s

Available columns: ['Sales_Method', 'Sales_Volume_Group', 'Replication', 'Operating_Profit']

	Sales_Method	Sales_Volume_Group	Replication	Operating_Profit
0	Online	Low	1	149.881266
1	Online	Low	2	154.980759
2	Online	Low	3	86.591550
3	Online	Low	4	114.502199
4	Online	Low	5	101.119999

Sales_Volume_Group	High	Low	Medium
Sales_Method			
Online	370.717331	112.890591	222.751165
Store	326.926149	101.168395	201.134823



19. Decision Making / Expected Value

Objective: Select the business alternative with the highest expected value.

Expected Value = Probability × Payoff. Expected-value components are summed by alternative.

```
decision = pd.read_csv(
    "Walmart_Decision_Making_Dataset.csv"
)

print(
    "Available columns:",
    decision.columns.tolist()
)

display(decision.head())

decision["Expected_Value_Component"] = (
    decision["Probability"] *
    decision["Payoff_INR"]
)

values = (
    decision
    .groupby("Alternative")["Expected_Value_Component"]
    .sum()
    .sort_values(ascending=False)
)

display(values)

print(
    "Best alternative:",
    values.idxmax()
)

print(
    "Expected value:",
    values.max()
)

```

✓ 0.0s

Available columns: ['Alternative', 'Scenario', 'Probability', 'Payoff_INR']

	Alternative	Scenario	Probability	Payoff_INR
0	Increase Online Advertising	Low	0.25	180000
1	Increase Online Advertising	Normal	0.50	320000
2	Increase Online Advertising	High	0.25	500000
3	Open New Store	Low	0.25	80000
4	Open New Store	Normal	0.50	280000

Alternative

Increase Online Advertising	330000.0
Open New Store	310000.0
Increase Discounts	285000.0

Name: Expected_Value_Component, dtype: float64

Best alternative: Increase Online Advertising

Expected value: 330000.0

20. Optimization

Objective: Maximize expected profit subject to limited budget, labor hours and product capacity.

A PuLP linear programming model uses product allocation decision variables and resource constraints.

```

optimization = pd.read_csv(
    "Walmart_Optimization_Input_Dataset.csv"
)

display(optimization)

print(
    "Products:",
    optimization["Product"].tolist()
)

# Linear programming model:
# Maximize total profit subject to budget, labor and product limits.

```

```
try:
```

```

    from pulp import (
        LpMaximize,
        LpProblem,
        LpVariable,
        lpSum,
        LpStatus,
        value
    )

```

```

    problem = LpProblem(
        "Walmart_Profit_Optimization",
        LpMaximize
    )

```

```
    product_vars = {}
```

```
    for _, row in optimization.iterrows():
```

```

        product_vars[row["Product"]] = LpVariable(
            row["Product"].replace(" ", "_"),
            lowBound=0,
            upBound=row["Max_Units"],
            cat="Integer"
        )

```

```

    # Resource limits for academic demonstration
    budget_limit = 5000000
    labor_limit = 1200

```

```

    problem += lpSum(
        product_vars[row["Product"]] *
        row["Profit_Per_Unit"]
        for _, row in optimization.iterrows()
    )

```

```

    problem += lpSum(
        product_vars[row["Product"]] *
        row["Budget_Per_Unit"]
        for _, row in optimization.iterrows()
    ) <= budget_limit

```

```
    problem += lpSum(
```

```

        problem = LpProblem(
            "Walmart_Profit_Optimization",
            LpMaximize
        )

```

```
    product_vars = {}
```

```
    for _, row in optimization.iterrows():
```

```

        product_vars[row["Product"]] = LpVariable(
            row["Product"].replace(" ", "_"),
            lowBound=0,
            upBound=row["Max_Units"],
            cat="Integer"
        )

```

```

    # Resource limits for academic demonstration
    budget_limit = 5000000
    labor_limit = 1200

```

```

    problem += lpSum(
        product_vars[row["Product"]] *
        row["Profit_Per_Unit"]
        for _, row in optimization.iterrows()
    )

```

```

    problem += lpSum(
        product_vars[row["Product"]] *
        row["Budget_Per_Unit"]
        for _, row in optimization.iterrows()
    ) <= budget_limit

```

```

    problem += lpSum(
        product_vars[row["Product"]] *
        row["Labor_Hours_Per_Unit"]
        for _, row in optimization.iterrows()
    ) <= labor_limit

```

```
    problem.solve()
```

```
    print("Optimization Status:", LpStatus[problem.status])
```

```
    print("Optimal Product Allocation:")
```

```

    for product, variable in product_vars.items():
        print(product, ":", variable.value())

```

```

    print(
        "Maximum Expected Profit:",
        value(problem.objective)
    )

```

```
except ImportError:
```

```

    print(
        "PuLP is not installed. Run: pip install pulp"
    )

```

```
0.0s
```

	Product	Profit_Per_Unit	Sales_Per_Unit	Labor_Hours_Per_Unit	Budget_Per_Unit	Max_Units
0	Laptop	8500	62000	2.5	48000	150
1	Smartphone	5200	42000	1.8	33000	220
2	Office Chair	2400	12000	1.2	8200	300
3	Printer Paper	450	1800	0.4	1200	700
4	Coffee	320	900	0.3	600	900
5	Skin Care	700	2500	0.5	1600	500

Products: ['Laptop', 'Smartphone', 'Office Chair', 'Printer Paper', 'Coffee', 'Skin Care']
 PuLP is not installed. Run: pip install pulp

21. Final Business Summary

The final KPI summary reports Total Sales, Total Profit, Total Units Sold, Average Discount, Average Customer Rating and Unique Customers.

```
print("WALMART BUSINESS ANALYTICS SUMMARY")
print("-----")

print(
    "Total Sales:",
    round(df["Sales"].sum(), 2)
)

print(
    "Total Profit:",
    round(df["Profit"].sum(), 2)
)

print(
    "Total Units Sold:",
    round(df["Quantity"].sum(), 2)
)

print(
    "Average Discount:",
    round(df["Discount"].mean() * 100, 2),
    "%"
)

print(
    "Average Customer Rating:",
    round(df["Customer_Rating"].mean(), 2)
)

print(
    "Unique Customers:",
    df["Customer_ID"].nunique()
)
```

✓ 0.0s

```
WALMART BUSINESS ANALYTICS SUMMARY
-----

Total Sales: 10720494.19
Total Profit: 1765316.54
Total Units Sold: 54773
Average Discount: 10.6 %
Average Customer Rating: 3.95
Unique Customers: 2454
```

22. Business Recommendations

- Focus on regions that produce stronger sales and profit while investigating underperforming regions.
- Improve product categories with lower profitability through pricing, inventory and promotion decisions.
- Monitor discount levels because excessive discounting can reduce profitability.
- Use monthly sales trends and moving averages for inventory and demand planning.
- Compare Online and Store performance before allocating additional resources.
- Use A/B testing before introducing major promotional campaigns.
- Use simulation to prepare for low, normal and high demand conditions.
- Use ANOVA to support decisions when comparing group performance.
- Use factorial experiments to understand combined effects of sales method and sales volume.
- Use expected value to compare uncertain business alternatives.
- Use optimization to allocate limited budget and labor toward higher-profit products.

23. Conclusion

This project applied Business Analytics techniques to a Walmart retail business dataset. The analysis progressed from data loading and preprocessing to descriptive statistics, exploratory visualization, correlation, regression, probability, sampling, confidence intervals, bootstrapping, hypothesis testing, A/B testing, time-series analysis, simulation, ANOVA, factorial experimentation, decision analysis and optimization.

The project demonstrates how raw retail transaction data can be converted into business insights that support decisions related to products, regions, sales channels, promotions, demand planning and resource allocation.

24. References

- Python documentation
- Pandas documentation
- NumPy documentation
- Matplotlib documentation
- Seaborn documentation
- SciPy documentation
- Scikit-learn documentation
- Statsmodels documentation
- PuLP documentation
- Business Analytics course material